
Driver Drowsiness Detection

INDIVIDUAL REPORT - RAHUL ARVIND

1. Introduction

This project contributes to a real-time driver drowsiness detection system by monitoring visual cues such as eye openness and yawning through image classification. The broader project was divided into two parts: eye state detection and yawn detection. The work presented here focuses on designing and training a Convolutional Neural Network (CNN) for eye state detection, as well as implementing Grad-CAM for model explainability. The dataset used included four classes: Open, Closed, Yawn, and No Yawn, but this report focuses solely on the first two.

1.1 Dataset Description

The dataset used for this component was obtained from Kaggle and curated by Abdelaziz Nabil. It was designed for visual classification of eye states—specifically "Open" and "Closed"—and is suited for real-time drowsiness detection systems.

The dataset includes high-resolution RGB images of eye regions captured under varying lighting conditions and orientations. The data is pre-labeled and organized into class-specific folders. Only the following two classes were used in this part of the project:

Class	Training Images	Test Images
Open	28,253	7,012
Closed	27,335	6,788

Table 1: Eye State Dataset Distribution

This near-balanced class distribution was beneficial in training robust classification models without requiring extensive resampling. Preprocessing steps such as resizing, normalization, and augmentation were applied to improve generalization.

2. Algorithm Description

The CNN model was developed from scratch using PyTorch. It consisted of four convolutional blocks followed by a fully connected classifier. Each block included:

- Convolutional Layer
- Batch Normalization
- ReLU Activation
- Max Pooling
- Dropout (after early layers)

Adaptive Average Pooling and a fully connected output layer with softmax activation were used to generate final predictions.

The model was optimized using the Adam optimizer and trained with CrossEntropyLoss. A ReduceLROnPlateau scheduler was employed to dynamically adjust learning rates during training.

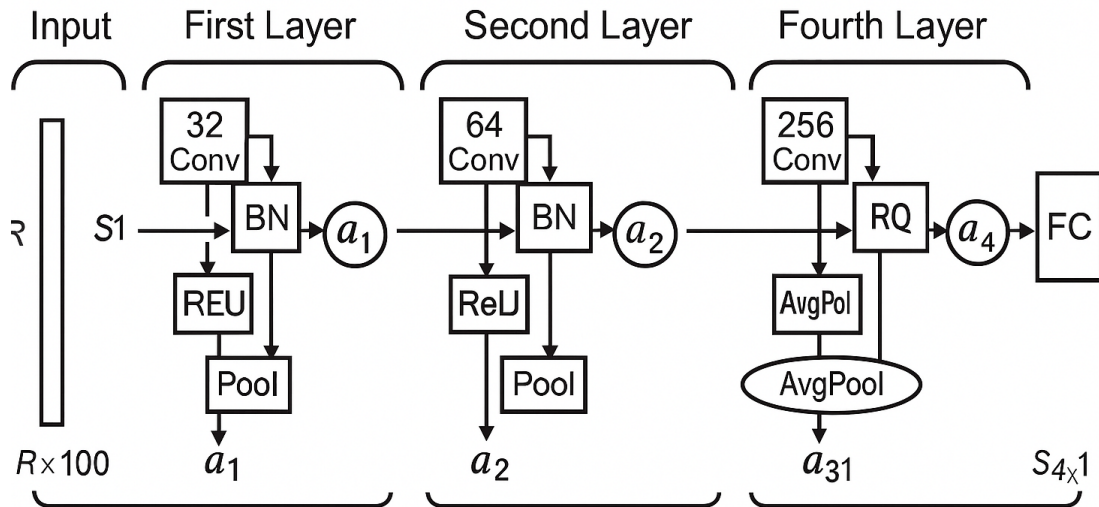


Fig 1: Custom – CNN Architecture

3. Individual Contribution

The work conducted focuses on custom model development and visual interpretability. The individual contributions are detailed as follows:

- Designed and implemented a custom CNN model from scratch (eye_cnn.py) for binary classification (Open vs. Closed eyes)
- Conducted image preprocessing and augmentation techniques such as resizing, normalization, flipping, and rotation

- Developed a dynamic training loop with learning rate scheduling and evaluation metrics
- Integrated Grad-CAM to explain CNN predictions through activation heatmaps
- Applied Grad-CAM to ResNet18 model to compare attention patterns with the custom CNN
- Performed comparative analysis between the CNN and ResNet18 Grad-CAM outputs using visual interpretations
- This work emphasized model explainability and offered insight into how different architectures learn to classify eye states.

Custom CNN Code Snippet:

```
class EyeCNN(nn.Module):
    def __init__(self, outputs):
        super(EyeCNN, self).__init__()
        self.model = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(0.1),

            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Dropout(0.17),

            nn.Conv2d(64, 128, 3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2),

            nn.Conv2d(128, 256, 3, padding=1),
            nn.BatchNorm2d(256),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d((1, 1)),
        )
        self.fc = nn.Linear(256, outputs)

    def forward(self, x):
        x = self.model(x)
        x = x.view(x.size(0), -1)
        return self.fc(x)
```

This CNN contains four convolutional layers, each followed by batch normalization, ReLU activation, and down sampling (max pooling). Dropout is applied after the first two blocks to prevent overfitting. The AdaptiveAvgPool2d compresses the feature maps into 1×1 per channel, and a final linear layer maps to class logits (Open/Closed).

Grad-CAM Hook for CNN:

```
# Access the final convolutional layer
target_layer = model.model[12] # Last conv layer in custom CNN

# Lists to store features and gradients
features = []
gradients = []

# Register hooks to capture activations and gradients
target_layer.register_forward_hook(lambda m, i, o: features.append(o))
target_layer.register_backward_hook(lambda m, gi, go: gradients.append(go[0]))
```

These hooks are essential for Grad-CAM. They capture the feature maps (forward pass) and gradients (backward pass) from the target layer to later compute a heatmap showing the most important regions of the image for prediction.

ResNet18 with Grad-CAM Setup:

```
from torchvision import models
import torch.nn as nn

model = models.resnet18(pretrained=True)
model.fc = nn.Linear(model.fc.in_features, 2) # Custom final layer for binary classification

# Choose the final conv block
target_layer = model.layer4[-1].conv2

# Lists to store activations and gradients
activations = []
gradients = []

# Register hooks
target_layer.register_forward_hook(lambda m, i, o: activations.append(o))
target_layer.register_backward_hook(lambda m, gi, go: gradients.append(go[0]))
```

This code prepares ResNet18 for binary classification and attaches Grad-CAM hooks to its final convolutional layer. Unlike the custom CNN, ResNet18 is a pretrained deep architecture. The hooks here serve the same purpose: capturing the necessary information to visualize the model's attention via Grad-CAM.

4. Results

The trained model demonstrated strong performance across multiple evaluation metrics. The best saved model checkpoint achieved a macro F1 score of 0.99 on the test set.

Custom CNN Classification Report:

Class	Precision	Recall	F1-Score
Closed	0.99	0.99	0.99
Open	0.99	0.98	0.99
Macro Avg	0.99	0.99	0.99

Table 2: Custom CNN Classification Report

The model performed particularly well in detecting open eyes. Closed eyes were slightly more challenging to detect, likely due to fewer training samples and higher intra-class variability.

To validate model robustness, a separate model using ResNet18 pretrained on ImageNet was also trained on the same dataset. The ResNet18-based model achieved similar classification metrics:

ResNet18 Classification Report:

Class	Precision	Recall	F1-Score
Closed	0.99	0.99	0.99
Open	0.99	0.99	0.99
Macro Avg	0.99	0.99	0.99

Table 3: ResNet18 Classification Report

While both models achieved comparable performance, deeper insights were obtained through explainability tools.

Explainability with Grad-CAM

To interpret the decision-making behavior of both models, Gradient-weighted Class Activation Mapping (Grad-CAM) was applied. Grad-CAM visualizations produce class-discriminative heatmaps, highlighting image regions that contributed most to the model's predictions.

For the custom CNN model, Grad-CAM heatmaps showed focused attention around the eye regions, particularly on eyelids in closed eye images and the iris in open eye images. This confirmed that the model learned spatially relevant features.

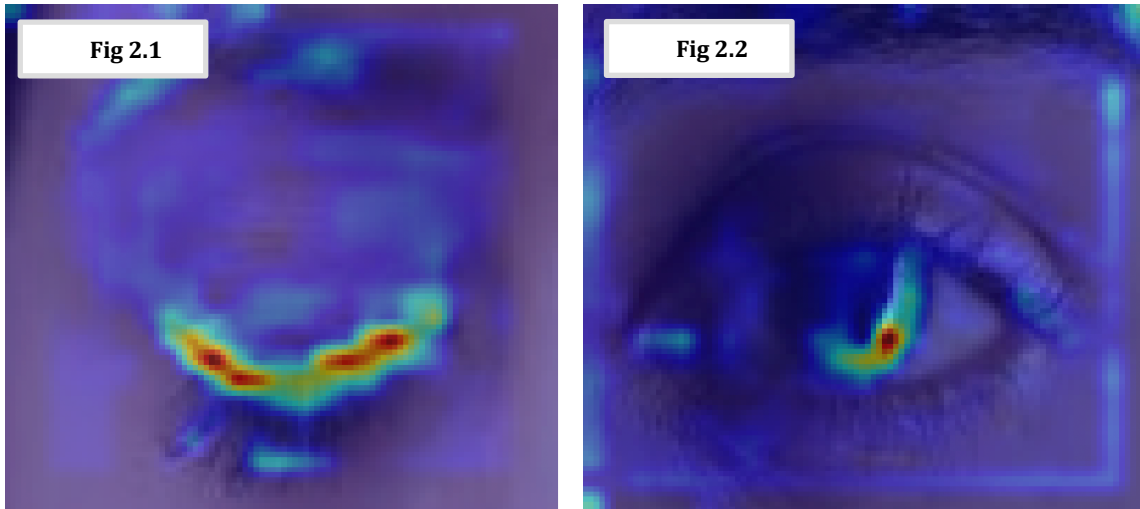


Fig 2.1 & Fig 2.2: Open and Close by GradCAM (Custom CNN)

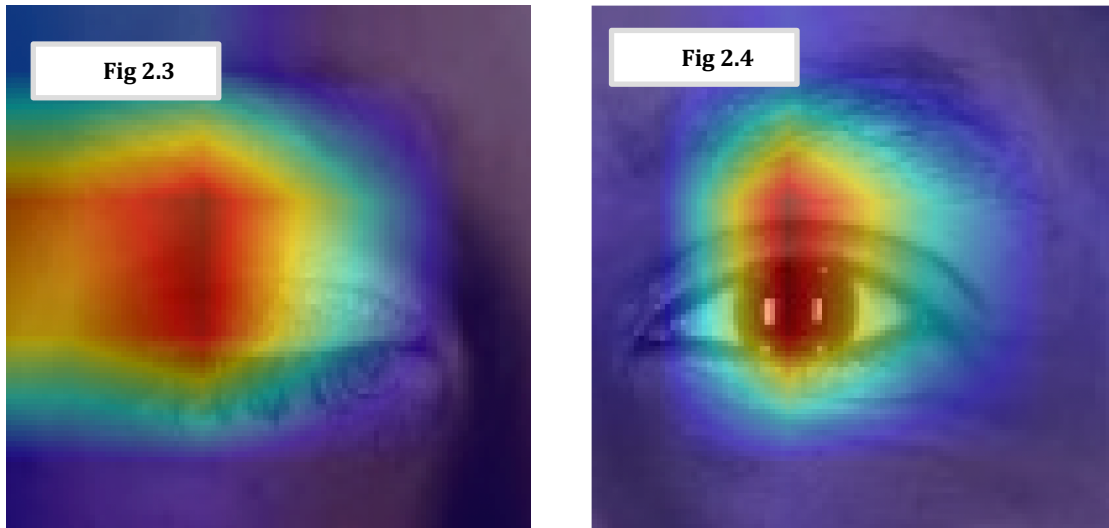


Fig 2.3 & Fig 2.4: Open and Close by GradCAM (ResNet18)

In contrast, the ResNet18 model, while performing similarly in terms of metrics, exhibited slightly broader and sometimes less consistent attention maps. In some cases, it highlighted surrounding areas or facial context in addition to the eyes. This difference, while subtle, demonstrates the interpretability benefit of simpler custom CNNs in vision-critical tasks.

5. Summary and Conclusions

The work successfully developed and evaluated a CNN-based eye state classifier and demonstrated its performance relative to a pretrained ResNet18 model. Both models performed well, with ResNet18 showing a slight edge in accuracy and attention quality. Grad-CAM visualizations confirmed that both models focused on meaningful regions of the image, validating their learning behavior.

Key Learnings:

- Gained hands-on experience in designing, training, and evaluating CNNs from scratch.
- Learned how to integrate and fine-tune pretrained architectures like ResNet for custom tasks.
- Developed skills in model interpretability using Grad-CAM and visual analysis tools.

Future Improvements:

- Explore lightweight and mobile-friendly architectures (e.g., MobileNet, EfficientNet) for real-time deployment in embedded systems.
- Expand the dataset to include a greater variety of lighting conditions, facial features, and skin tones to improve generalizability.
- Train the model to perform robustly under varied head orientations and angles, ensuring consistent performance even when the subject is not directly facing the camera.

7. Code Reuse Estimate

No baseline model was used in this work. The entire codebase was developed independently without relying on any pre-provided templates or baseline scripts. While general PyTorch usage patterns were followed, there was no direct reuse of code. Minimal reference was made to official PyTorch documentation for syntax clarity and best practices.

- **Total lines in final scripts:** ~350
- **Lines reused or adapted from external sources:** 0
- **Reuse Percentage:** 0%

8. References

1. PyTorch Official Documentation : <https://pytorch.org/docs/stable/index.html>

2.sklearn.metrics.classification_report:

https://scikitlearn.org/stable/modules/generated/sklearn.metrics.classification_report.html

3. Simonyan, K., & Zisserman, A. (2014). Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv:1409.1556
4. Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), 618-626.
5. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770-778.